7.1.1 Conditional Resampling Loop. Consider the continuous program depicted in Figure 15, containing continuous sampling, unbounded recursion, and conditioning inside of the recursion. Slice automatically discretizes this program, also shown in Figure 15. We generate the corresponding Markov chain by unfolding the small-step semantics in a breadth-first manner. In the presence of unbounded recursion, this Markov chain *might* be infinite and this process might thus diverge. If, however, it terminates, the resulting Markov chain is finite and precisely captures the execution behavior of the discretized program. For this example, this is the case: Even though the discretized program contains unbounded recursion, its execution behavior can be modelled as a finite-state Markov chain[2], where $\bot$ is a special state entered when encountered an observation failure.

~~Now, using~~ Using the probabilistic model checker Storm, we can ~~perform exact inference: It is immediate that the Markov chain's probability $\Pr(\Diamond \texttt{true} \mid \Box\neg\bot)$ of eventually reaching~~ compute three distinct quantities. First, the raw extended semantics records the mass of returning true~~(in LTL: $\Diamond$true) *given* that we never enter the observation-failure state~~, the mass of returning false, and the mass of observation failure $\bot$~~(in LTL: $\Box\neg\bot$) equals the measure of~~; its missing mass is divergence. Second, conditioning on observation success divides by $1 - \Pr(\Diamond\bot)$. This retains divergent paths and is therefore not, in general, a distribution over returned values. Third, the normalized posterior over returned values divides by the probability of eventual successful termination:

$$\Pr_{\text{post}}(\text{true}) = \frac{\Pr(\Diamond\text{true})}{\Pr(\Diamond\text{true}) + \Pr(\Diamond\text{false})},$$

provided that the denominator is positive.

The distinction matters for this example. Resolving the 0.2 resampling self-loop gives raw masses $\Pr(\Diamond\text{true}) = 0.375$ and $\Pr(\Diamond\bot) = 0.25$, and divergence mass 0.375. Thus conditioning only on avoidance of $\bot$ gives $\Pr(\Diamond\text{true} \mid \Box\neg\bot) = 0.375/0.75 = 0.5$, because $\Box\neg\bot$ includes the divergent executions. The program has no false-returning execution, so its normalized posterior over returned values instead assigns probability $0.375/0.375 = 1$ to true~~in the program's normalized posterior distribution~~.

7.1.2 Random Interval Covering of $[0, 1]$ *with Conditioning.* Our second case study extends the random interval covering example from §2.6 by conditioning inside of the recursion: We condition on *every* sample falling within *at most* one interval. The difference to the program in §2.6 is:

```
1  let non_overlap = fix non_overlap y :=
2    fun seen -> fun ys ->
3      match ys with
4      | nil -> true
5      | h :: t ->
6          let hit = (snd h) y in
7          if hit && seen then false
8          else non_overlap y (seen || hit) t
9      end in
11 let cover = fix cover st :=
12   if uniform(0,1) < 0.1 then st
13   else
14     let y = gaussian(0,1) in
15     let _ = observe(non_overlap y false st) in
16     cover (step y st)
17 in
18 let final = cover init in
19 is_covered final
```

The key new component is the recursive function `non_overlap` (l. 1), which checks whether a sampled point y hits more than one interval of the initially sampled intervals. The loop `cover` (l. 11) repeatedly samples a Gaussian point and immediately conditions on this predicate via **`observe`** (l. 15), so only executions in which the point lies in at most one interval are retained. The measure of true in the conditional posterior distribution is thus the probability that this unbounded process hits every interval at least once *given* that each sampled point falls in at most one interval.

[2]We apply a straightforward optimization: Intermediate execution steps that do not branch probabilistically can be collapsed into a single transition. This is the reason why all states in Figure 15 correspond to the header of the recursive function.

Continuous Program

```
1  (fix resample x :=
2    let _ = observe(x >= 0.2) in
3    if x <= 0.5 then true
4    else if x <= 0.8 then resample x
5    else
6      let y = uniform(0,1) in
7        resample y) 1
```

Discretized Program

```
9  (fix resample x :=
10   let _ = observe(x >= 1) in
11   if x <= 1 then true
12   else if x <= 2 then resample x
13   else
14     let y = discrete(0.2,0.3,0.3,0.2) in
15       resample y) 3
```

Markov Chain Construction & Model Checking

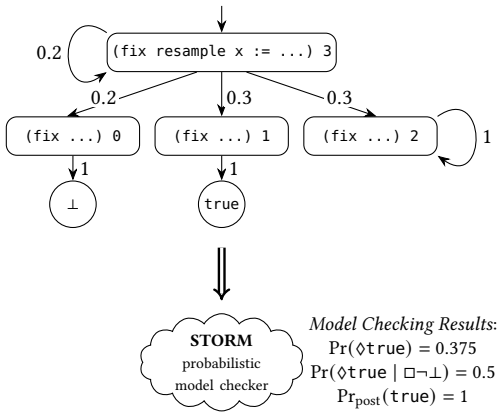


Fig. 15. Conditional resampling loop and its discretization. The continuous program is transformed into a finite-state discrete program (Slice alone takes less than 0.001s), which induces a finite Markov chain (generated in 0.001s) and can then be analyzed by Storm (with a model checking time of 0.024s). The ⊥-state in that Markov chain is entered when encountering an observation failure.

The program remains discretizable: each sampled point is still used only through comparisons against finitely many sampled interval boundaries, and `non_overlap` depends only on the resulting hit pattern, not on the exact real value of y. Slice replaces each Gaussian draw by its interval index, yielding a discrete program (shown in Figure 22) with a finite exact state space. Slice takes 0.001s, and we generate a Markov chain consisting of 268878 states in 80 seconds. Storm takes 4.7 seconds for model checking, and reports that the sought-after probability is 0.000227....

Currently, our Markov chain construction process is rather naive: extending the number of sampling points or intervals yields the Markov chain to be infeasibly large. Developing efficient symbolic discrete inference engines for higher-order programs with unbounded recursion is not in the scope of this paper. However, our approach is an excellent motivation for this future direction.

7.1.3 Bloom Filter. We model a probabilistically populated variant of a Bloom filter [8], which is a data structure using hash functions for storing data space efficiently: one can insert data into a Bloom filter and subsequently query whether a given data point is stored in it. Due to aggressive compression via hashing, this process can be faulty: while it can never produce false-negatives, it can produce false-positives. Now conisder the following program (full program shown in Figure 23):

```
      else
        if perfect <= 0 then 0
        else
          discrete(1.0, 0.0, 0.0) in
let _ = observe(gpa <= 1 && gpa >= 1)
nationality > 0
```

After discretization, the equality test `gpa = 10` becomes the discrete test that the GPA index is exactly the one corresponding to the singleton region $\{10\}$.

A.4 Point and Measure-Zero Observations

Slice can condition on an exact value when that event has strictly positive probability mass, as in the Indian GPA example in §A.3. In that case, ~~conditioning~~ returned-value normalization is well-defined and Slice ~~returns the correct answer~~preserves the raw extended semantics from which the posterior is computed. The exact set of programs supported by Slice is therefore somewhat larger than one would initially expect, while not covering full point observations on continuous samples.

The mechanism is the same as in §A.3: the cuts $<c$ and $\leq c$ induce the partition

$$(-\infty, c), \qquad \{c\}, \qquad (c, \infty),$$

so any probability mass assigned exactly to c is tracked as its own discrete outcome.

More precisely:

(1) Slice cannot handle programs where every control-flow path conditions on an event of measure zero, and therefore programs with exact observations on continuous samples are in general out of scope, i.e. the programs one typically writes in e.g. Stan are out of scope unless one widens exact conditioning to a small but finite interval.
(2) Slice can handle programs where at least one control-flow path conditions only on strictly-positive-probability events, even if that is a point observation, and even if other paths condition only on measure-zero events.
(3) It does not matter whether an event is measure-zero because it is point-conditioning on a continuous sample, or whether it is measure-zero for another reason, e.g. **observe**(**uniform**$(0,1) > 3$) or simply **observe**(false). The critical distinction for **observe**(E) is whether $\Pr(E) = 0$, not whether E is of the form "$x = 3.2$".
(4) Slice will never give the wrong answer, even when all paths condition on measure-zero events. It will give the empty subdistribution, indicating that under a rejection-sampling interpretation, all samples are rejected with probability 1. The normalized posterior is then undefined because the successful-return mass is zero. Thus the user is notified, and Slice never silently gives a wrong answer.

For instance, this program:

```
let x = uniform(0,1) in
if flip() then
  let _ = observe(x >= 0.5 && x <= 0.5) in true
else
  let _ = observe(x <= 0.3 && x >= 0.3) in false
```

~~results in:~~ has raw extended-semantic masses:

$$\Pr\mu(\text{true}) = 0 \qquad \Pr\mu(\text{false}) = 0.$$

Consequently, the returned-value posterior is undefined, rather than a distribution assigning probability zero to both values.

Whereas this program:

```
let x = uniform(0,1) in
if flip() then
  let _ = observe(x >= 0.5 && x <= 0.5) in true
else
  let _ = observe(x <= 0.3001 && x >= 0.3) in false
```

~~results in:~~ has raw extended-semantic masses

$$\Pr\mu(\text{true}) = 0, \qquad \Pr\mu(\text{false}) = 1.0.5 \cdot 0.0001 = 0.00005, \qquad \mu(\bot) = 0.99995.$$

~~This is the correct behavior~~Only after normalizing over successful returns do we obtain $\Pr_{\text{post}}(\text{false}) = 1$. This agrees with rejection sampling: consider repeatedly running the program ~~under a rejection-sampling interpretation~~ until an observation succeeds. With probability 1, the first successful test will be **observe**$(x \leq 0.3001$ && $x \geq 0.3)$ and the program will therefore return false with probability 1.

A.5 Random Interval Covering Over $[0, 1]$

The discretized program for the case study from §2.6 is shown in Figure 20. Slice takes 0.000s, and we generate a Markov chain consisting of 192365 states in 38.69 seconds. Storm takes 0.69 seconds for model checking, and reports that the sought-after probability is 0.034474....

B Slice Case Studies (continued)

B.1 Random Interval Covering Over $[0, 1]$ with Conditioning

The full program for the case study from §7.1.2 is shown in Figure 21. The discretized program is shown in Figure 22.

B.2 Bloom Filter

The full program for the case study from §7.1.3 is shown in Figure 23. The discretized program is shown in Figure 24.

C List of Supported Slice Distributions

The list of distributions supported by Slice, along with their probability density / mass functions, is given in the following table. Slice supports discrete distributions by applying the same cut-set and value-set analysis used for continuous distributions, *provided that the discrete distribution admits a well-defined CDF*. The extension to discrete distributions does not introduce any fundamental changes to the discretization algorithm.

D Typing Rules for Full Slice Language

The typing rules for general language constructs is given in Figure 25.

E Discretization Proofs

E.1 Proof of Lemma 5.4

Theorem E.1 (Discretization Is Well-Typed). *Let* $\Gamma \vdash e : \tau$. *We have* $\mathcal{D}[\![\Gamma]\!] \vdash \mathcal{D}[\![e : \tau]\!] : \mathcal{D}[\![\tau]\!]$, *where* $\mathcal{D}[\![\Gamma]\!]$ *transforms all types in* Γ, *i.e.,* $\mathcal{D}[\![\Gamma]\!](x) = \mathcal{D}[\![\Gamma(x)]\!]$ *for all* $x \in \text{dom}(\Gamma)$.

Proof. We proceed by induction on the structure of the typing derivation $\Gamma \vdash e : \tau$, matching each case of the discretization function $\mathcal{D}[\![\cdot]\!]$ defined in Figure 13.